Trenchscan: phone video to 3D, tested on a radiator

Jakob Wirén · October 2026 · Code and notebook: github.com/jakowiren/trenchscan. The notebook (`notebooks/01_sparse_reconstruction.ipynb`, executed copy with all plots in `notebooks/01_sparse_reconstruction.html`) has the code and plots behind this report, and references like §2.4 point to its sections.

1 Goal

sizeless documents underground utilities from a phone video. A crew films an open trench with an iPhone Pro for about ten minutes, and sizeless turns that into a metric 3D model, recognises the pipes, fittings and couplings, and delivers CAD and GIS as-built plans (website, Y Combinator page). So for fun I spent Sunday on the first step myself, turning a phone video of a radiator at home into a 3D model.

2 Setup

I used a white column radiator in my apartment as the stand-in for a trench, with its vertical tubes playing the pipes. It shares a lot with a trench. I filmed it on a modest phone (Oppo A38, 720p max), there are no markers on anything, furniture and a corner limit how far back I can stand, and the surfaces are smooth and uniform. The right end is against the wall in a corner, so I could not film it, and I already expected the model to be weak on that side.



Figure 1. The three clips, 36–47 s each: top-down along the radiator with bank cards on the floor, a side view of the left end, and an arc from the left end round to the front.

I measured the radiator with a tape. The tubes are 28.6 mm in diameter and 55 mm apart centre to centre, and the radiator is 960 mm tall. The bank cards were meant as a scale reference, but I have not used them yet (Section 7).

3 Theory

Here is what happens from the moment someone films the trench until there is a 3D model.

- **Filming creates parallax.** The person filming walks along the trench, so every frame shows the same pipes from a slightly different position. Parallax is how much an object seems to shift between two views taken from different places. Near things shift more than far things, like your finger does when you hold it up and close one eye and then the other. This shift is what tells us the depth. So the person filming has to actually walk along the trench while filming. The distance the phone moves between two frames is called the baseline. Standing still and panning the phone gives almost no depth at all.

- **Features link the frames.** To measure that shift, the software has to find the same spot in different frames. It looks for features, small distinctive patches like a corner, a speck or a printed letter. The classic detector, SIFT, finds spots that stand out from their surroundings and describes each one by the pattern of brightness around it, so it can find it again in the next frame. A smooth surface with soft shading has no such spots, so on a clean white pipe there is nothing for it to hold on to.
- **Geometry finds the cameras and the points.** Some of the matches will be wrong. All the correct matches between two frames have to agree with one camera movement (epipolar geometry), so a voting method called RANSAC keeps the matches that agree and throws away the rest. Structure-from-motion (SfM, here COLMAP) then works out where the phone was for each frame and where each matched spot is in 3D, by intersecting the lines of sight from two or more frames (triangulation). It adds frames one at a time and places each new one using 3D points it can already see. At the end, bundle adjustment tweaks all cameras and points together so that every 3D point lands where it was seen in the images. What is left over, in pixels, is the reprojection error, and if it is well under a pixel the model is consistent. sizeless does the same thing live while filming, which is called SLAM (simultaneous localisation and mapping).
- **Scale is missing.** From images alone you cannot tell a small pipe filmed close up from a big pipe filmed from further away, since scaling the whole scene and the camera path together gives exactly the same images (Hartley and Zisserman). So the model gets the right shape but has no units, until something of known size is in view, like a card, or a sensor measures how far the phone moved in metres.
- **Measuring the video.** I use two numbers to describe each frame. Sharpness is the variance of the Laplacian, a filter that reacts to fine detail. Blur smears out detail, so a blurry frame gets a low score. Image speed is how fast the picture moves across the screen in pixels per second, which I get from optical flow (Farnebäck), an estimate of how far each pixel moved between two frames.

4 What I found

4.1 Looking at the data first (§1.1–1.4)

Before reconstructing anything, I went through the clips.

- Only the left side and the front are covered, which makes sense since the right end is in the corner. The bank cards are in clip 1 and early in clip 2, but not in clip 3, and my feet are in most of clip 1.
- I expected the blurry frames to come from moving the phone fast, but sharpness and image speed are barely related in any of the clips (§1.3). The softest stretch of clip 2 is when the phone is almost still and pointing at the bare wall. So the sharpness score cannot tell a blurry frame from a frame that just has nothing in it.
- I picked clip 3 for the reconstruction. It moves along the radiator instead of turning, which gives parallax, and it keeps the most radiator in view without getting as close as clip 2.

4.2 The reconstruction (§2.1–2.4)

87 of the 107 frames that passed the blur filter registered into one model, with 3,594 points and a reprojection error of 0.66 px, so where it works the geometry is consistent (more numbers in §2.2). Figure 2 shows the model. The camera path is a smooth arc at about the same distance the whole way, which matches how I filmed it. The points draw the outline of the radiator, its left edge and bands along the top and bottom, but where the tubes are the model is almost empty.

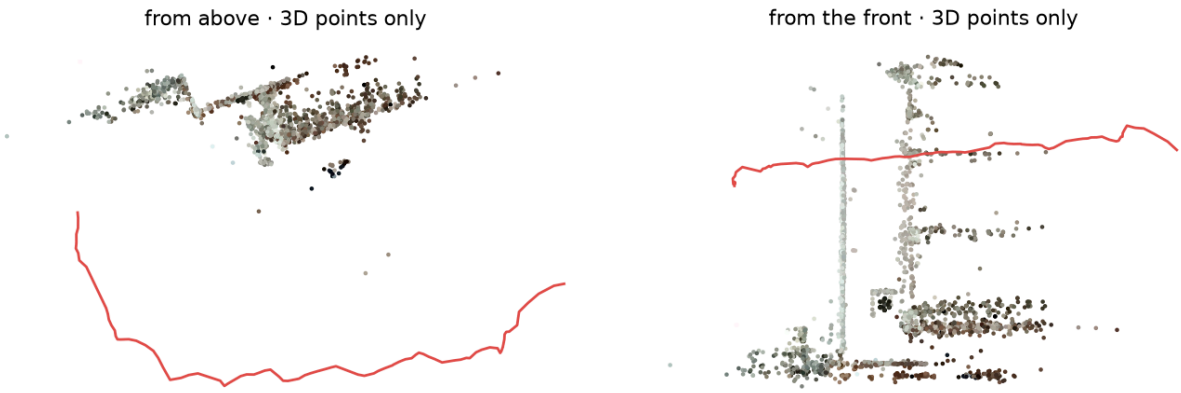


Figure 2. The 3D model of clip 3 from above and from the front. The red line is the camera path.

To see why, I looked at where the 3D points are in the images (Figure 3). The regions with no fine detail, in blue, cover 57 % of an average frame, but only 4 % of the 3D points are in them (§2.4). The points sit on edges, the gaps between tubes, the floor and the cards, and the white tube faces are basically empty. This is the smooth surface problem from Section 3 showing up in practice.

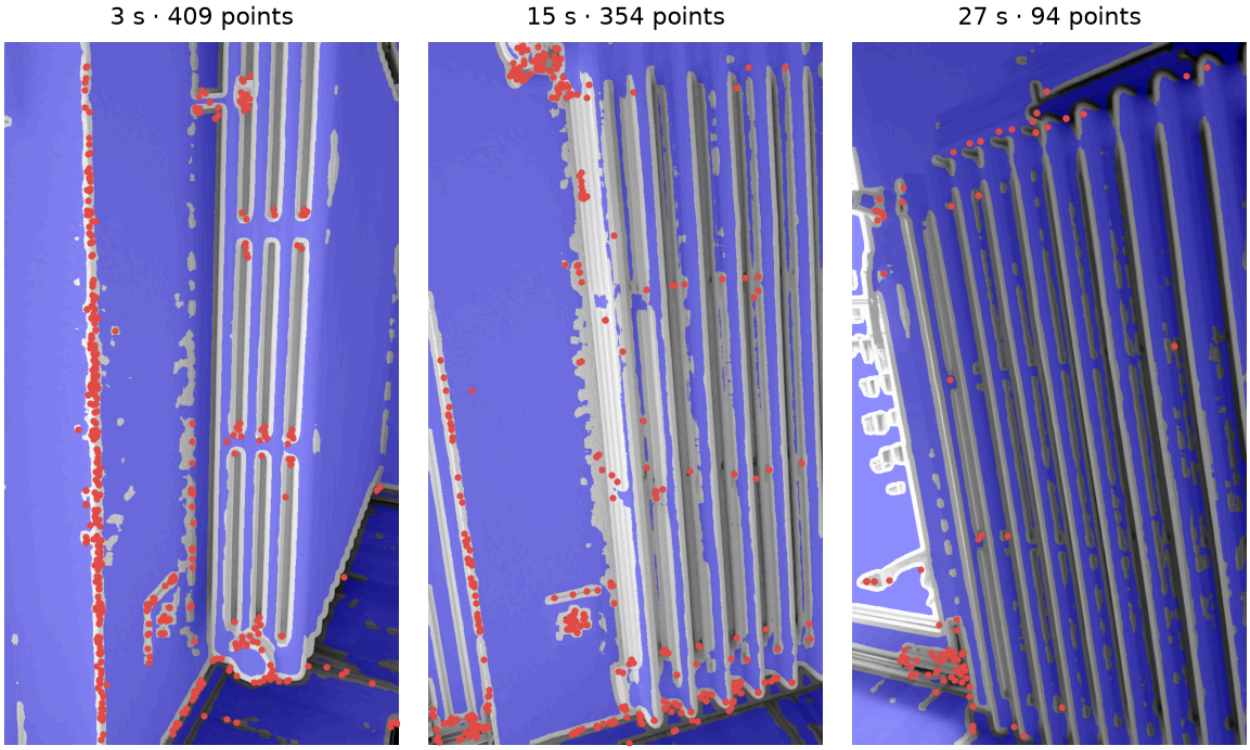


Figure 3. Three frames of clip 3. Blue: regions without fine detail. Red: features that became 3D points.

I also wanted to know which frames failed (Figure 4). All 20 frames that did not register are from the fast close-up at the end of the clip, and once the chain of frames breaks there it never recovers. The number of 3D points per frame drops from about 390 to about 90 as the image speed goes up (rank correlation −0.62). Speed alone does not explain it, though. Some of the failed frames are slower than frames that registered fine, and in this clip I also got closer as I moved faster, so I cannot tell which of the two caused it.

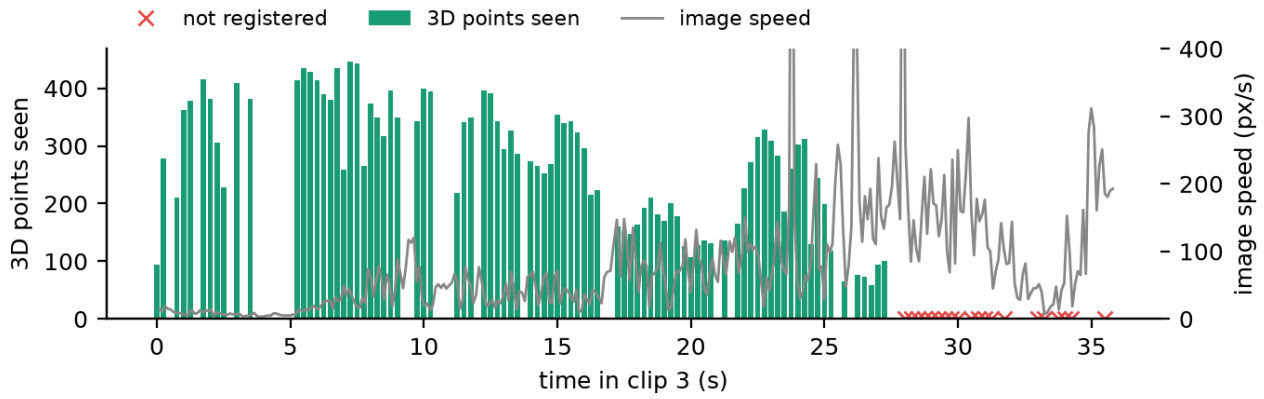


Figure 4. 3D points seen in each frame of clip 3, the frames that failed to register, and the image speed (capped at 400 px/s).

The blur filter had its own problem. Many of the frames it threw out are from the slow start, where the view is mostly plain wall, which fits what I saw in 4.1 (§2.1).

5 Trying lines instead of points (§3.1–3.4)

Looking at Figure 3 again, the tubes are very easy to see in the images. What makes them visible are the long, straight, dark gaps between them. SIFT wants spots that stand out in every direction, and a point on a straight edge looks just like the next point along the same edge (this is called the aperture problem). So it only finds points where an edge ends or turns. A whole line, on the other hand, is easy to find. So I thought, what if I just replace the points with lines and see what happens?

To keep it quick, I reused what already worked and made one assumption.

- I kept the camera positions from SfM, since that part worked well.
- I found the lines with LSD, a standard line detector.
- I assumed the tubes are vertical. Seen from above, a vertical line is just a point. A line in one frame, together with where the phone was, makes a flat plane, and seen from above that plane is a ray going out from the phone through the line. The rays of the same tube from many frames all cross at the same spot. So if I count how many rays pass through each spot on a grid (a Hough-style vote), the tubes should show up as dots. The nice thing is that I never have to decide which line in one frame belongs to which line in another. Every line just votes on its own, and the tubes show up where many of them agree.

There are proper tools that map lines in any direction (Line3D++, LIMAP), but they take more setup. This shortcut was a few notebook cells, which was enough to test the idea.

Figure 5 shows the lines on the same frames as Figure 3. They follow the tube edges and gaps in every frame, also in the frames where only a few 3D points survived.

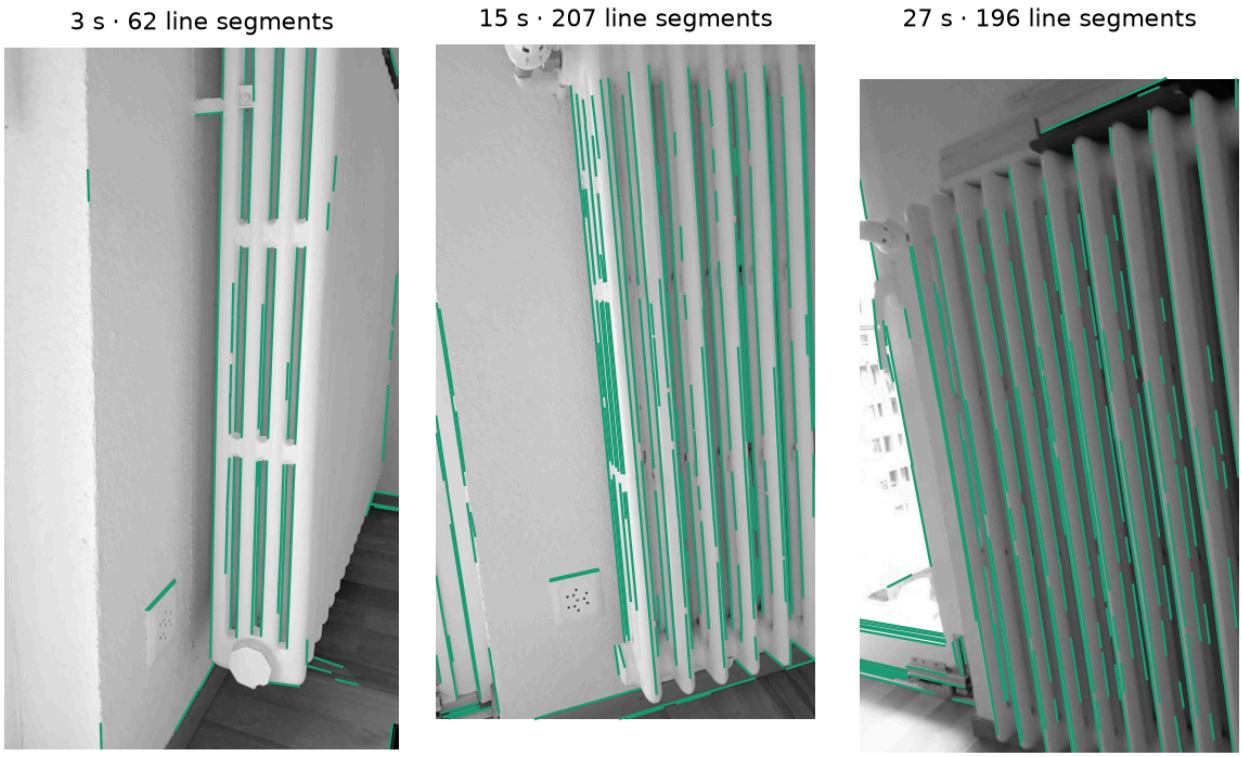


Figure 5. The same three frames as Figure 3, with the line segments LSD finds.

Figure 6 is the vote seen from above, and the tubes show up as a row of evenly spaced dots along the radiator front. The dots repeat every 0.36–0.37 model units, which the 3D points barely do (§3.3). The lines also start and stop sharply at the top and bottom of the tubes, 6.0–6.2 units apart. Spacing divided by height does not depend on scale, and it gives 56–58 mm for the 960 mm radiator, against the 55 mm I measured with the tape.

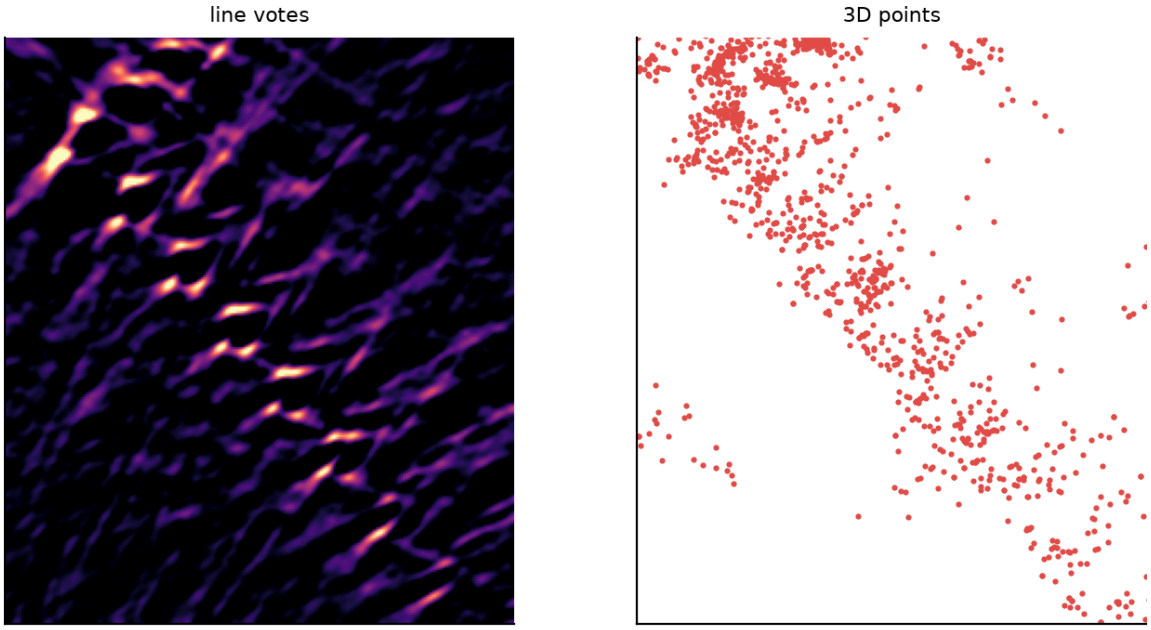


Figure 6. The radiator front seen from above, at the same zoom. Left: the vote, where each bright dot is a vertical line that many frames agree on. Right: the 3D points from SfM. The phone was off to the left, outside the picture, so the dots are stretched towards it. From where the phone was, it is easier to tell left from right than near from far.

Figure 7 puts both approaches in one model. On top are the 3D points alone, and below the same points with a vertical line for each dot, running from where its segments start to where they end. The lines fill in the radiator front that the points left empty, and their ends line up with the top and bottom bands of points. There are more lines than tubes though, 18 lines over 7.3 tube spacings, since each tube gives a line for its edges and the gaps next to it.

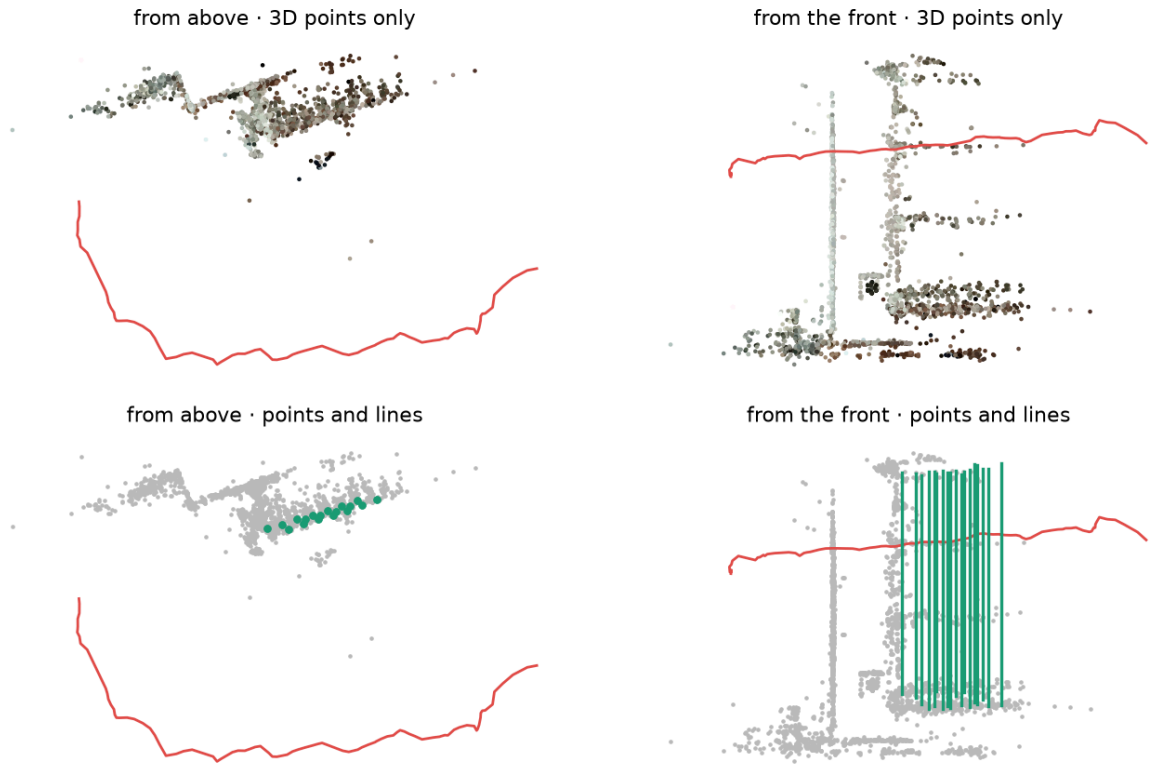


Figure 7. Top: the 3D model from Figure 2. Bottom: the same points in grey, with the vertical lines from the vote in green.

So the tubes are in the video as lines, and the points miss most of them. What I get is positions and not diameters yet. A tube's visible edge is where your line of sight just touches the cylinder, and that moves when the camera moves, so a diameter needs a cylinder fitted to those edges. It also only works because the tubes are vertical. Pipes in a trench mostly run horizontally along it, so the vote would have to run along the pipe direction instead, or a general line mapper would take over.

6 Discussion: what this means for a product like sizeless

1. **Poses are easy, the objects are hard.** The camera path is reasonable, but the pipes themselves, smooth and uniform, carry almost no points and therefore feature extraction is hard. In a trench, PVC, painted steel and water behave the same way. sizeless reports pipe diameters as an output, and our evidence says surface points from classical features would not support that. Shape has to come from edges and silhouettes, as the lines in Section 5 do for position, or from methods that do not depend on texture.
2. **Guide the capture with image speed.** Speed probably correlates with the loss of points (Section 4.2), while the sharpness score does not, and it discards sharp frames of bare surfaces. A capture app can measure optical flow live and warn before the frames are lost.
3. **Coverage is decided on site.** The unfilmed corner cannot be recovered later. Once a trench is backfilled, neither can the trench, so the app has to show what is missing while the crew is still there.
4. **Scale must be built in.** Our model has no units. sizeless captures with an iPhone Pro and builds a metric model in its SLAM engine. A card or something of known size in view is only a fallback, and only if it is in the clip that gets reconstructed, which it was not in this case.
5. **Real scenes move.** My feet already disturb clip 1. sizeless names wet, muddy trenches with moving excavators as what breaks off-the-shelf reconstruction.

Step	sizeless	This demo
Capture	Guided iPhone Pro app, about 10 min per trench, works without GPS	Unguided 36–47 s clips from a 720p phone
3D	Metric 3D model from a SLAM engine	Sparse SfM, about 3,600 points, no scale, static scene
Components	Pipe types, diameters, fittings, couplings	None
Measurements	Measurements and billable quantities	Tube spacing relative to height, from lines (Section 5); tape ground truth recorded
Output	CAD (DWG/DXF), BIM/GIS, crew voice notes anchored in 3D	Interactive 3D view

Sources for sizeless: website, YC page, App Store listing.

7 Future work

Each of these follows from something I found, and I can test it with data I already have.

Finding	Next step	Evaluation
No scale (§2.2)	Scale from the bank card; combine clip 1, rotated, with clip 3	Tube spacing 55 mm and height 960 mm against the tape
Smooth tubes hold 4 % of points (§2.4)	Learned matching (SuperPoint + LightGlue) or learned reconstruction (MASt3R, VGGT)	Point share in smooth regions
Lines find the tubes, but only vertical ones and no diameter (§3.3)	Cylinders fitted to the tube edges; line mapping in any direction (LIMAP), tried on horizontal pipes	Diameter 28.6 mm; spacing 55 mm
Points fall as speed rises; the blur score drops sharp, bare frames (§2.1, §2.4)	Live speed warning during capture; select frames by speed and overlap instead of sharpness	Share of frames that register; points per frame
Moving objects in view (§1.2)	Mask people before matching	Reprojection error and outliers with and without masks

8 Limitations

This is one radiator, filmed in one session on one phone, so everything here comes from a single scene. A radiator in a living room is also a lot easier than a trench. It is dry, indoors, and nothing moves except my feet.

Some things I could not separate with this data. In clip 3 I got closer as I moved faster, so I cannot say if speed or distance broke the model. The smooth-region measure covers wall and tube faces together, because nothing segments the radiator yet, and its threshold was chosen by eye, so §2.4 checks a range of thresholds. The line trial only works because the tubes are vertical, and the spacing result assumes the lines run the full height of the radiator. COLMAP also gives slightly different point counts from run to run, so the exact numbers are from one run. Everything I say about sizeless comes from their public pages, not from using the product.

The obvious next step is to just capture more data, and learn by trial and error what types of data make reconstruction better.